Parcial Algoritmos y Programación II - 7541 Curso Calvo (1º instancia) 26 de octubre de 2017

Padrón: **Apellido y Nombre:**

Punteros: APROBADO – DESAPROBADO **TDA:** APROBADO – DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

  int *A, *C, *F;
  int **B;
  char *D, *E;
  char G;
  int H;

  H = 64 + ULTIMO_DIGITO_PADRON;
  G = 'A';
  A = &H;
  D = &G;
  B = &A;
  cout << *A <<  *D << **B << endl;

  C = new int[3];
  F = C + 2;
  C[1] = H;
  (*A) = 70;
  (*C) = H;
  (*F) = 68;
  C[2]++;
  cout << C[2] << C[1] << C[0] << endl;

  (**B) = C[0] + 1;
  A = C + 2;
  (**B) = H;
  C[0] = *A;
  (*F) = A[0] - 3;
  cout << C[2] << C[1] << C[0] << endl;

  E = (char*) A;
  C[1] = (**B) - 4;
  cout << (*D) << (*E) << G << endl;

  // liberar la memoria
  return 0;
}
```

2. Implementar para la clase Lista<T> con una estructura simplemente enlazada el siguiente método, indicando pre y post:

```
void insertar(unsigned int posicion, T elemento);
```

3. Implementar el método `buscarMesaConDiferenciaMaximaEntreDosCandidatos` de la clase `EstadisticoElectoral` a partir de las siguientes especificaciones:

```
class EstadisticoElectoral {

  public:
    /* post: procesa 'mesas' y devuelve la MesaElectoral en la que existe la máxima diferencia entre los votos
     *       recibidos por 'unCandidato'  y 'otroCandidato'.
     *       Solo considera aquellas Mesas Electorales cuya cantidad de votos es menor o igual a la cantidad de
     *       votantes de su padrón.
     */
    MesaElectoral* buscarMesaConDiferenciaMaximaEntreDosCandidatos(List<MesaElectoral*>* mesas,
                                                                   Candidato unCandidato, Candidato otroCandidato);
}
```

```
class MesaElectoral {

  public:
    /* post: Mesa identificada por 'número' y sin Votos.
     */
    MesaElectoral(int numero);

    /* post: devuelve el número que identifica la Mesa.
     */
    int obtenerNumero();

    /* post: devuelve todos los Votos recibidos.
     */
    Lista<Voto*>* obtenerVotos();

    /* post: devuelve todos números de documento de los votantes
     *       habilitados para votar en esta Mesa.
     */
    Lista<int>* obtenerPadron();

    /* post: libera todos los Votos acumulados.
     */
    ~MesaElectoral();
};
```

```
class Voto {

  public:
    /* post: voto para el Candidato indicado.
     */
    Voto(Candidato candidatoVotado);

    /* post: voto en blanco.
     */
    Voto();

    bool esBlanco();

    /* pre : no es un voto en blanco.
     * post: devuelve el Candidato votado.
     */
    Candidato obtenerCandidato();
}

enum Candidato {

  CANDIDATO_1, CANDIDATO_2, CANDIDATO_3
  , …, CANDIDATO_N;
}
```

4. Diseñar la especificación e implementar el TDA Préstamo. Debe proveer operaciones para:

- crear el Préstamo recibiendo como parámetro su monto [$].
- calcularSaldo: devuelve el monto [$] que falta pagar.
- pagar: recibe el monto [$] a pagar y lo descuenta del saldo.
- contarPagos: devuelve la cantidad de pagos realizados.
- calcularPagoMaximo: devuelve el monto [$] del máximo pago realizado.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas

Solución 1:

```
**0**
64A64
696470
686471
ADA
**1**
65A65
696570
686571
ADA
**2**
66A66
696670
686671
ADA
**3**
67A67
696770
686771
ADA
**4**
68A68
696870
686871
ADA
**5**
69A69
696970
686971
ADA
**6**
70A70
697070
687071
ADA
**7**
71A71
697170
687171
ADA
**8**
72A72
697270
687271
ADA
**9**
73A73
697370
687371
ADA
```

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas